Decoys That Do Work: Evaluating Executed Cover for LLM Agent Communication Privacy

***Abstract*—LLM Agents can hide which tool operation a user requested by sending decoy calls. When an observer sees the hop to a third-party tool and the privacy target includes which request–response exchanges occur and how long the service takes, TLS and HTTP/2 padding cannot imitate responses, so decoys must reach the application. We show that such cover leaks through the process that dispatches it. Honest tool calls follow a model decision; decoys inserted by a controller do not wait for one, and this difference is exploitable on real TLS records. In 3,000 single-host executions with two fixed-weight planners over MCP and TLS 1.3/HTTP/2, label-free timing removal raises recognition of the requested action from 75 to 112 (Qwen) and from 80 to 108 (Llama) of 120 tasks when call boundaries are supplied. Dispatching decoys after sampled historical gaps defeats that removal, yet adapted attacks still gain over size-only attacks in three of four comparisons after Holm correction, recognizing 93–106/120. A shadow planner on an independent ticket brings the strongest tested attack to 70/120 and 64/120, consistent with a 7/12 plausible-deniability reference, at twice the model tokens and with more and different cover calls. We define an evaluation that binds inference, task completion, business-state integrity and cost to the same executions, and state which service capabilities each policy assumes.**

1. Introduction

An LLM Agent receives a support ticket, checks an order and submits a refund through a third-party commerce service. The traffic is encrypted, yet an observer of the path to that service sees which endpoints are contacted, how large each exchange is and how long the Agent waits between them. Recent work shows that such metadata reveal prompts, tasks, workflows and user traits [2], [3], [4], [9].

A defender who wants to hide the refund can add decoy calls. Classical traffic-analysis defenses discard their cover at a cooperating endpoint before it reaches an application [34]. On the hop to a third-party service, the defender can still pad TLS records and HTTP/2 frames, which the service's protocol stack discards [25], [24]. Padding shapes the defender's requests, but it cannot make the service answer. If the observer may learn which request–response exchanges occur and how long the service takes, decoys must reach the application and elicit real responses. Such *executed cover* has two outputs: a communication trace and application work, which can consume quota, touch other objects and re-enter the planner's context.

The central observation. Executed cover leaks through the process that sends it. An honest tool call is issued after the model decides on it, so it follows a model wait. A decoy inserted by the controller does not have to wait for anything. We show that this difference is exploitable on real TLS records, that hiding it by sampling historical gaps leaves the signal largely intact, and that the natural alternative, a second planner that generates cover by running a complete independent task, changes the trace at a measurable cost. Decoy-query obfuscation in web search taught a related lesson, that decoys are only as good as their generator [63], [64], [65]. In Agents the generator is coupled to model inference, and decoys perform business work.

Approach. We state when cover must reach the application, classify executed cover along four axes (termination, state, feedback and dispatcher), and list the service capabilities each policy assumes. We define an evaluation that binds residual inference, requested-goal completion, business-state integrity and resource cost to the same protected executions. We apply it to 3,000 live executions of two fixed-weight planners, Qwen3:8B and Llama3.1:8B, over MCP and TLS 1.3/HTTP/2 on one host, comparing four cover dispatchers against no cover with a cooperating tool provider.

Findings.

- **Controller dispatch is exploitable.** Given call boundaries, label-free gap removal raises recognition of the requested action from 75 to 112 (Qwen) and from 80 to 108 (Llama) of 120 tasks; both gains survive Holm correction, and refitting on independent packet-capture clocks reproduces them.
- **History matching hides the shortcut, not the signal.** Dispatching after sampled historical gaps defeats the removal, yet adapted attacks still gain over size-only attacks in three of four comparisons after Holm correction, recognizing 93–106/120.
- **Without call boundaries the picture changes.** The prespecified removal attack loses its advantage on complete encrypted connections, while recognition stays well above chance.
- **A shadow planner lowers recognition at a cost.** The strongest tested attack reaches 70/120 and 64/120, consistent with a 7/12 plausible-deniability reference; the shadow doubles model tokens and adds more and different cover calls, so the gain is not yet attributable to planner generation alone.

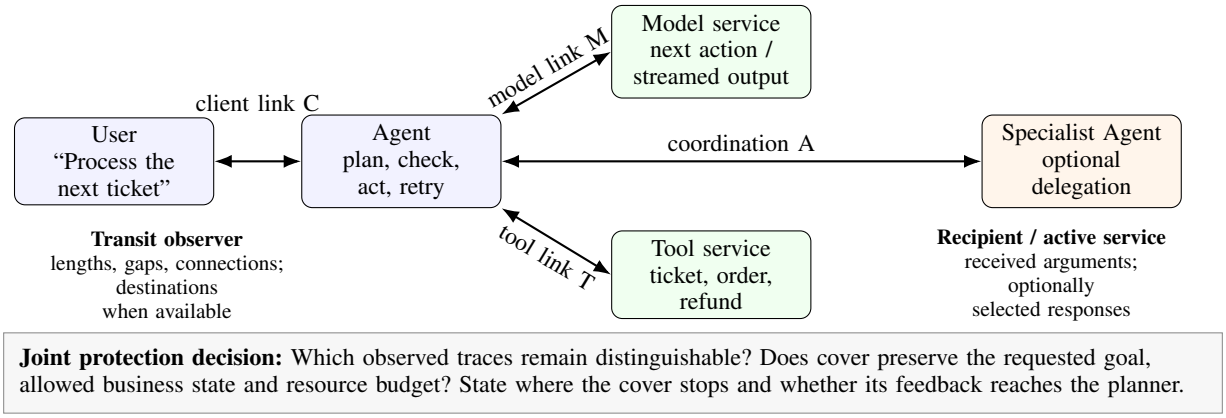


Figure 1. Running support-ticket example and observation rights. A tool result returns to the Agent and can enter the next model request. Each encrypted link exposes its own traffic shape; a recipient additionally sees the request addressed to it. Concurrent tickets may share a connection on either link. The arrows represent application communication, not a one-to-one mapping from tool events to packets.

- **Goal completion is not state integrity.** In a stress control where cover writes to live state, 95 of 102 completed Qwen tasks leave collateral damage that a goal check misses.

Contributions. (1) The dispatcher as a leakage source for executed cover, with live evidence on TLS records (Sections 6–7). (2) A threat model that states when cover must reach the application, a four-axis design space and the service contract each policy assumes (Sections 3–4). (3) A joint evaluation protocol with interval-based decisions and the plausible-deniability reference for planner cover (Section 5). Appendix A maps all forty-four existing Agent side-channel settings; in the four query/workflow privacy settings among them, none reports both an adapted attacker and original-task utility under the same intervention.

2. Background and Motivation

2.1. Agent communication

An Agent uses an LLM to choose actions, and external feedback can change its later actions or termination. Figure 1 shows the links of the running example: client–Agent, Agent–model, Agent–tool and inter-agent coordination. A tool result returns to the Agent and can enter the next model request. Tools are commonly reached through the Model Context Protocol (MCP), either over local stdio or over Streamable HTTP [23]. Remote connections typically use TLS 1.3 and HTTP/2, which encrypt content but expose record lengths, directions and timing, and multiplex several request streams on one connection [25], [24]. A captured packet, TLS record, HTTP stream, tool call and complete task are therefore different units, and a boundary supplied by an evaluator is prior information, not an attacker output.

2.2. Communication side channels in Agents

A small but growing literature shows that Agent traffic leaks. Transit observers recover research prompts and user traits from contacted domains and timing [2], workflow classes from inter-agent communication graphs [3], and tasks or misuse from model-link and egress traffic [4], [9], [1]. Contacted recipients fingerprint Agent clients and models from protocol headers, terminal commands and tool-call arguments [6], [7], [48], [47], [35], and active defenders plant lures for Agent adversaries [10], [51]. Appendix A maps all forty-four settings from fifteen such studies by observer access, target, unit and intervention. Most evaluate recognition; few evaluate a protection.

2.3. How traffic defenses are evaluated

Website-fingerprinting (WF) work has established what a defense evaluation must contain: realistic observer information and splits, attacks adapted to the protected distribution, and overheads measured on the same traffic [15], [16], [32]. Adaptive-attack and ML-security guidance make the same demands [19], [52]. Classical defenses pad, pace or add dummy traffic, and their cover never reaches an application [17], [18], [66], [34]. State-aware Agent benchmarks check environment outcomes [60], [61] but do not measure traffic.

2.4. The gap

Table 1 applies those requirements to the four Agent query/workflow privacy settings we located. WRA's five research decoys lower query recovery from .770 to .565, but its report-utility scores of 8.55 and 8.54 belong to a different intervention that changes destination visibility [2]. The communication-graph study refits its attacker on live OHTTP captures and reports about thirteen-fold bytes, but no original-task outcome under that intervention [3]. No setting reports both an adapted attacker and matched utility, and the only executed-cover setting reports no matched utility at all. This is a statement about four settings from two sources, not a field-wide frequency; it shows that the

measurements a deployment decision needs have not yet been made together.

3. Threat Model

Assets and adversary. The private target is the user's requested task, here one of six support actions. The defender controls the Agent and its controller, and may run relays, but does not control third-party tool services. We consider three observers, distinguished by their rights. A *transit observer* sees connection destinations, directions, ciphertext lengths and times on the links it watches. A *recipient* additionally reads the requests addressed to it, including arguments. An *active auditor* also chooses probes. Identifying a requested action does not establish whether it was authorized; that needs the principal and policy context. Table 2 maps these rights to common deployments and to the existing evidence for each view.

When cover must reach the application. Transport cover ends at an endpoint the defender controls, after the observer and before the application [34]. A relay outside a monitored egress can serve as that endpoint, but the hop from the relay to a third-party service remains. On that hop, the service's own protocol stack still offers two terminators the defender can use without the provider's cooperation. TLS 1.3 lets the sender pad any record and send padding-only records, which the receiver's TLS layer discards [25, Sec. 5.4]; HTTP/2 lets the sender pad DATA and HEADERS frames and send PING frames, which the receiver's HTTP layer consumes [24, Secs. 6.1, 6.7]. These mechanisms can shape the defender's own request bytes and timing. They cannot make the service send a response: a padding record elicits nothing, and a PING elicits only an immediate eight-octet acknowledgement. Response presence, size and service latency are produced by the application.

Whether cover must reach the application therefore depends on the privacy target. If the observer may learn which request–response exchanges occur, their response sizes or their service latencies, protocol-level padding leaves those signals and cover must elicit real application responses (Table 3). If only request-side shape matters, padding suffices. A recipient that operates the service sees every exchange it receives, so against it cover must also carry plausible arguments. Reaching the application does not by itself require a business write: a read-only or no-op call elicits a real response. Cover must perform a state-changing operation only if such operations produce responses or latencies that reads cannot imitate. We call any cover that reaches the application *executed cover*; the design space in Section 4 separates how much business work it performs.

Observer in this paper. Our measurements use a transit observer on the final hop to seven tool services, with no defender relay in between, and a privacy target that includes the exchange sequence and response timing. It receives destinations, ciphertext bytes and TLS record completion times. Section 7.1 additionally supplies episode and call windows, an evaluator privilege that Section 7.2 withdraws. The recipient view, where decoy arguments are also visible, is part of the threat model but not of these measurements (Section 8). The adversary may train on traffic generated under the same cover policy; the attack suite therefore includes classifiers and cover-removal models fitted on protected training executions, alongside attacks frozen on unprotected traffic.

4. The Executed-Cover Design Space

Four axes. We classify a cover policy along four axes (Table 4). *Termination* and *state* determine what business work cover performs: a disposable copy, read-only access, an isolated clone or production state. *Feedback visibility* determines whether cover responses enter later model requests and can change the real planner's decisions. The *dispatcher*, the process that decides when cover is sent, determines whether cover inherits the timing of honest calls. Honest calls follow a model decision; a controller that inserts cover without waiting for the model creates a timing signature that the first three axes do not capture. Our live study varies the dispatcher while holding termination (an isolated clone) and feedback (hidden) fixed; an earlier state control varies termination and feedback.

The execution contract. Because executed cover performs work, a policy must satisfy a contract as well as a privacy target. The contract names the objects and transitions cover may touch, the quota, token and latency budgets it may consume, and the feedback it may expose to the real planner. Isolation prevents direct writes to production state but still consumes compute and can change scheduling; a response hidden from the planner does not undo the operation that produced it. A requested goal can complete while another order, a shared quota or a ledger is wrong, so the contract separates three outcomes: requested-goal completion, violations of declared business invariants, and resource use. For a synthetic fixture whose intended transition is fully specified, exact state equality is the invariant; in production, invariants must admit legitimate concurrent updates.

The service contract. Every policy also needs capabilities from the service (Table 5). Our live study gives the defender a *cooperating provider*: services pad response bodies, hold a cover response for a sampled minimum duration, and execute cover on an isolated clone. This matches a provider that offers a privacy mode or a sandbox on the same endpoint, while its network path is observed by others. Such a provider could also discard cover in its application and return padded, delayed dummy responses, which is the proxy-discard row of Table 4; our dispatcher result applies to that design too, because it concerns when cover requests are sent, not what the service does with them. A non-cooperating provider offers none of the service-side capabilities: the observer additionally sees unpadded response sizes and unheld response timing, and cover executes against real state. We therefore read our leakage results as measured under defender-favorable service conditions, and we do not measure the non-cooperating case.

Where executed cover sits among defenses. Name hiding and relays change destination association; padding

TABLE 1. EVIDENCE LOCATED IN THE RETAINED RECORDS. COUNTS USE ELIGIBLE SETTINGS; THE SECOND COLUMN ALSO GIVES DISTINCT SOURCES. "NOT LOCATED" LEAVES EVIDENCE UNRESOLVED WITHIN THE INSPECTED SCOPE. THESE COUNTS DESCRIBE AVAILABILITY, NOT STUDY QUALITY OR CHECKLIST VALIDITY.

Question	Eligible settings / sources	Located	Supporting settings and consequence
Protected-condition attack adaptation	4 / 2	2/4	Communication graphs: transformed captures and live OHTTP. Both come from one preprint.
Original-task utility under the same intervention	4 / 2	1/4	WRA visibility comparison. It does not price WRA's executed-decoy condition.
Adaptation and original-task utility in one setting	4 / 2	0/4	Adaptation and utility are located in different records; transport cost alone does not fill the task-outcome field.
Original-task utility for executed cover	1 / 1	0/1	WRA's five research decoys report recovery and extra research; matched original-task utility is not located.

TABLE 2. DEPLOYMENT AND ACCESS MAP. THE CONFIGURATIONS ARE ANALYST-DEFINED; CITED SETTINGS TEST THE LISTED VIEW, NOT EVERY COMPONENT OF THE DEPLOYMENT. A GATEWAY'S PLAINTEXT ACCESS REQUIRES TLS TERMINATION.

Deployment	Observer and permitted view	Existing evidence for that view	Protection consequence
Enterprise egress	A transit monitor sees routed model/tool flows, sizes, times and any exposed destination association.	WRA: supplied research-domain traces; ANT and cloud benchmark: model/combined traffic [2], [9], [4].	Specify which services cross the monitored egress. A union of flows is available only when the monitor can observe them.
Shared gateway or relay	Transit sees multiplexed gateway output. A terminating gateway can additionally read addressed requests.	Live OHTTP: passive-link inference under stated relay trust [3]; PrivacyTrace: semantic recipient/gateway access [35].	Separate relay association protection from content trust. Connection sharing changes the observation unit.
Local model, remote tools	An external observer sees remote tool traffic; the local model call has no corresponding external network leg.	WRA's destination-recovery mechanism is relevant to tool egress [2]; whole-deployment protection is untested in the mapped comparison.	Evaluate tool and downstream calls. Do not import a remote-model attack score into an absent network leg.
Cloud model, remote tools	An authorized vantage may see either or both encrypted legs. A contacted tool sees its own arguments.	ANT/cloud benchmark test model traffic [9], [4]; PrivacyTrace tests recipient-visible representations [35].	Check permitted model and tool views after the intervention; recipients and passive monitors need different protections.

TABLE 3. WHAT EACH TERMINATOR HIDES FROM AN OBSERVER OF THE FINAL HOP TO A THIRD-PARTY SERVICE. PROTOCOL-LEVEL PADDING IS DISCARDED BY THE SERVICE'S TLS OR HTTP/2 LAYER WITHOUT ITS COOPERATION [25], [24], BUT RESPONSES STILL FOLLOW REAL APPLICATION CALLS.

Cover mechanism	Terminated by	Still visible to the observer
Defender tunnel or relay	Defender endpoint	Everything on the relay's upstream hop
TLS record padding, padding-only records	Service TLS layer	Response presence, size and latency of real calls
HTTP/2 frame padding, PING	Service HTTP/2 layer	As above; PING answers are immediate 8-octet ACKs
Read-only or no-op call	Application	Read-versus-write response differences; quota use
Business operation (sandbox or live)	Application	Only what the attack can still separate

changes lengths; pacing changes timing [17], [18], [66]. These layers remain useful, and Table 3 shows when transport cover suffices. Executed cover is the remaining option when the observed hop ends at a service the defender does not control. Model services add their own signals, such as token streams, cache behavior and token counts [20], [12], [11], [13], [21]; a tool-side cover policy does not shape later model requests. Appendix B summarizes what each defense layer requires the defender to control, the costs to measure together and the exposure that remains.

5. Evaluation Methodology

5.1. Joint evaluation protocol

Privacy, task success, business state and cost must refer to the same intervention and evaluation population. The protocol has four steps; Table 6 is its reporting form.

1) **Specify observations and execution authority.** Name the hidden target, the visible links, recipient rights and any supplied boundaries. State where cover ends, which state it can access and whether its responses reach the planner.

TABLE 4. COVER TAXONOMY. EACH POLICY IS CLASSIFIED BY WHERE IT TERMINATES AND WHAT STATE IT TOUCHES, WHETHER ITS RESPONSES REACH THE REAL PLANNER, AND WHICH PROCESS DISPATCHES IT. THE LAST COLUMN NAMES THE EVALUATION IN WHICH THE POLICY IS MEASURED; S1 IS THE EARLIER STATE CONTROL AND SECTION 7.1 THE LIVE TIMING STUDY.

Policy	Terminates at / state touched	Feedback to planner	Dispatcher	Measured in
Transport cover	Tunnel endpoint before the service; no business state	No	Sender schedule	STP [34]; WF padding [66], [18]
Proxy discard	Defender proxy computes on a disposable copy	No	Controller	S1
Read cover	Real service; reads only	No	Controller	S1
Isolated execution	Real service on a persistent clone	No / yes	Controller	S1 (hidden and visible); all live cover arms
Live write	Real service on production state	No	Controller	S1 (stress control)
Research decoys	Real third-party Web destinations	Not located	Decoy generator	WRA [2]
Immediate	Isolated clone	No	Controller, directly after an honest response	Section 7.1 (calibration)
History-matched	Isolated clone	No	Controller, training-gap and duration samples; sequential or concurrent	Section 7.1
Planner shadow	Isolated clone; separate planner context	No	Second planner on an independent ticket	Section 7.1

TABLE 5. SERVICE CAPABILITIES USED BY THE LIVE STUDY AND WHO MUST PROVIDE THEM. THE STUDY THEREFORE MODELS A *cooperating provider*; AGAINST A NON-COOPERATING PROVIDER ONLY CLIENT-SIDE CAPABILITIES REMAIN.

Capability	Used by	Provided by	Non-cooperating provider
Response body padding (16 KiB quanta)	All arms	Service	Unavailable
Server-side hold of a cover response	History arms	Service	Unavailable; client can delay only its next request
Isolated clone for cover execution	All cover arms	Service	Only with a sandbox on the same endpoint
Request padding, dispatch timing	All arms	Client	Available
Same-service serialization	All arms	Client	Available

TABLE 6. JOINT EVALUATION CHECKLIST FOR ONE INTERVENTION AND POPULATION. ATTACH EVIDENCE OR STATE THE MISSING ITEM; GOAL COMPLETION AND WHOLE-STATE CORRECTNESS ARE SEPARATE FIELDS.

☐	Hidden target, population, prior or base rate
☐	Allowed observations and supplied boundaries
☐	Cover termination, privileges and isolation
☐	Planner-visible cover feedback and dispatcher
☐	Declared tolerances: inference, goal, state, tokens, bytes, latency
☐	Adapted attacks and largest tested residual
☐	Original-task attempts, completion and failures
☐	Business invariants and violations
☐	Matched bytes, latency, tokens and quota

2) **Specify the contract and tolerances.** Define the requested goal, allowed collateral transitions, and the inference, quota, token and latency tolerances, before measuring.
3) **Measure matched executions and adapted attacks.** Pair baseline and policy on the same workload. Include cover-removal and schedule-aware attacks, report the strongest tested residual inference, and take task, state and cost outcomes from those protected runs.
4) **Decide with uncertainty.** A policy fails if a declared tolerance is exceeded beyond sampling uncertainty, passes if every measured quantity is within its tolerance beyond that uncertainty, and is otherwise undecided. An execution outside its state or resource contract fails even when its goal passes. Every observer compared must face the same attack suite. Passing a finite suite is not a secrecy proof.

5.2. Attacks and statistics

Each condition has its own attack suite, fitted on its training split. Size-only attacks are logistic regression (LR) and a 300-tree random forest (RF) on destinations and ciphertext sizes, a classifier frozen on no-cover traffic, and last- and penultimate-destination rules. Timing attacks add timing features, supervised and label-free gap removal, context-aware removal and joint decoding; Supplement S2.3 defines them. The following conventions apply to every result. *Sel.* is the attack chosen on validation data before test scoring, and primary tests use only these choices. *Max* is the largest test count in a finite suite; it is descriptive and is not an upper bound on inference. *Exploratory* marks an analysis designed after test results were seen. Paired tests are exact McNemar tests with Holm correction within each prespecified family;

TABLE 7. PAIR-AMBIGUITY FLOOR $P^*(k)$ FOR k SHADOW PLANNERS AND SIX CLASSES, COMPUTED EXACTLY (EXPERIMENT/SHADOW_FLOOR.PY). *iid*: SHADOW LABELS DRAWN UNIFORMLY; *distinct*: DRAWN WITHOUT REPLACEMENT FROM CLASSES OTHER THAN THE REAL ONE. TOKENS ARE NOMINAL PLANNER MULTIPLES. ONLY $k = 1$, IID, WAS RUN.

k	iid	distinct	Tokens×	Live result ($k = 1$)
0	1.000	–	1	
1	.583	.500	2	Max 70 / 64 of 120
2	.491	.333	3	
3	.456	.250	4	
4	.428	.200	5	
Realized $k = 1$ floor on the test split: 70.5/120 (21 matching labels)				

intervals use 10,000 action-stratified fixture bootstraps and condition on the fitted attacks and designed workload.

5.3. A floor for planner cover

This floor is the plausible-deniability argument of decoy-query obfuscation: GooPIR hides a query among $k-1$ decoys [67], and an attacker who cannot tell them apart can do no better than chance among the candidates. We state it for Agent shadows because a shadow is a complete execution with its own label, so repeated labels change the bound. Planner cover runs k complete shadow executions beside the real one. Consider an idealized attacker who separates all $k+1$ chains perfectly and recovers each chain's label, but cannot tell which chain is real. If the real chain is exchangeable with the shadows, then given the multiset of labels the real label is c with probability $n_c/(k+1)$, where n_c counts the chains labeled c. The best guess is a modal label, which succeeds with probability

$$P^*(k) = \frac{\mathbb{E}[\max_c n_c]}{k+1}.$$

With one shadow and six uniform classes, $P^*(1) = \frac{1}{6} + \frac{5}{6} \cdot \frac{1}{2} = \frac{7}{12}$. Table 7 gives exact values. Drawing shadow labels without replacement from the classes other than the real one makes all labels distinct and gives $1/(k+1)$, while keeping each shadow's marginal label uniform. $P^*(k)$ is a reference, not a bound: an attacker who learns which chain is real exceeds it, and reused real fixtures with their own cache histories make exchangeability an idealization. The right comparison for a shadow policy is therefore $P^*(k)$, not the uniform $1/6$.

6. Experimental Setup

Workload and planners. Each fixture is a synthetic support ticket requesting one of six business actions over a state with three orders, a customer address, a compensation quota, an escalation status and a ledger. LangGraph [56] alternates model decisions with real tool calls to seven MCP Streamable HTTP services over TLS 1.3/HTTP/2, with request and response bodies padded to 4,096 and 16,384 bytes. Two fixed-weight planners, Qwen3:8B and

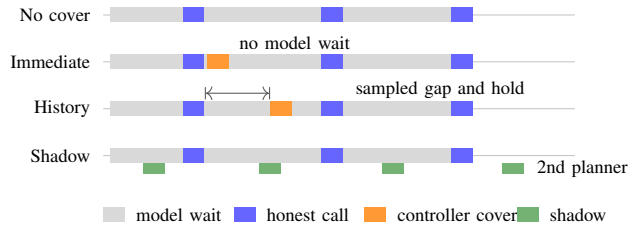


Figure 2. Dispatchers in the live study. Honest calls follow a model wait. Immediate cover follows the previous response without one, which label-free gap removal exploits. History-matched cover samples a training gap and response hold. Shadow calls come from a second planner, so their gaps are model waits too.

TABLE 8. STUDY MAP. EACH POPULATION ANSWERS ONE QUESTION AND KEEPS ITS OWN DENOMINATORS AND STATISTICAL ROLE.

Study	Population; observer	Question; role
Live timing (§7.1)	3,000 episodes + 600 shadow chains; call windows, TLS record times	Does the dispatcher expose cover? Primary families P, R, C
Record-only (§7.2)	Same episodes; no call boundaries	How much survives without supplied boundaries? Retrospective; one primary pair
State control (§7.4; S1)	3,600 episodes; call windows, sizes only	Does goal completion miss collateral state? Primary state family
Forwarding pilot (S10)	3,120 local + 192 hosted	Feedback attribution; inconclusive, supplement only

Llama3.1:8B, run at temperature zero with weights pinned by layer hashes. Each planner executes 300 fixtures per condition: 120 training, 60 validation and 120 paired test fixtures, balanced over the six actions. The label is the requested action, and wrong or failed executions remain in every denominator.

Cover policies. All four cover arms execute cover on an isolated clone and hide its responses from the real planner; they differ only in the dispatcher (Figure 2). *Immediate* cover follows an honest response directly. *History* cover samples a training-only honest gap and service duration, waits in real time and holds the response for the sampled minimum; the *sequential* variant blocks the real chain and the *concurrent* variant overlaps real model inference. *Shadow* runs a complete second planner on an independent ticket whose action is drawn uniformly, and a fair draw decides which chain starts first. Calls to the same service serialize. The five conditions give 3,000 main episodes and 600 shadow chains.

Studies. Table 8 lists the populations. The live timing study and its record-only reanalysis answer the privacy questions. A separate earlier state control on the same workload varies termination and feedback across six policies, including a deliberately harmful live-write arm, to test whether goal completion misses collateral state. A forwarding pilot appears only in Supplement S10. All runs use one host over

loopback with synthetic tools; Section 8 states what this does not test.

7. Results

7.1. The dispatcher leaks

Immediate cover is removable. A label-free two-component mixture over training gaps selects the short-gap component for removal, and a classifier trained on no-cover traffic decodes the rest. On immediate cover this restores 112/120 and 108/120 correct predictions, against selected size-only attacks at 75 and 80 (Table 9); both gains meet the prespecified replication rule after Holm correction ($p = 1.9\times10^{-8}$ and 4.3×10^{-6}). Independent TCP reassembly matches all 773,450 recorded TLS record types, lengths and counts, and refitting the mixture on packet-capture clocks reproduces both scores; on held-out immediate calls it flags 181/182 and 182/182 decoys with no false flags (Supplement S2.7). A controller that skips the model wait therefore exposes its cover.

History matching hides the shortcut, not the signal. Sampling honest gaps defeats the simple mixture: it falls to 53 and 14 (sequential) and 38 and 66 (concurrent). Adapted context removal or joint decoding still recognizes 93/120 and 104/120 sequential executions, against selected size-only attacks at 76 and 82 (Holm $p = .0559$ and $.00078$), and 99/120 and 106/120 concurrent ones against 66 and 68 (both $p < 10^{-4}$). The policy controls minimum waits, not achieved timing (Table 10): Qwen's mean sampled hold of 40.5 ms becomes 100.4 ms sequential and 171.2 ms concurrent at the record observer.

A failed simple attack is therefore not evidence of concealment. Llama's sequential mixture deletes 477/634 honest calls while detecting only 64/182 cover calls, which explains its 14/120, whereas joint decoding on the same traces reaches 104/120.

7.2. Without call boundaries

We reanalyze all 3,000 episodes from complete encrypted connections, including setup and teardown records. The attacker receives relative record times, directions, ciphertext lengths and destinations, but no call boundaries, operation names, role labels or HTTP stream assignment; whole-episode association remains supplied. An idle-gap rule proposes events (five gaps from 5 to 250 ms compete on validation accuracy), raw-record classifiers skip eventization, and a training-only mixture removes short-gap events. The immediate-arm pair is the only primary test. This suite does not include the context-removal and joint decoders of Section 7.1.

Selected removal recognizes 69/120 and 86/120 immediate-cover tasks against baselines of 80 and 90, paired differences of -9.2 and -3.3 points (Holm $p = .025$ and $.54$): the call-window removal advantage does not carry over. An exploratory eventizer, designed after these results were seen, uses the public padding quantum and MCP setup sequence to propose requests; it raises Qwen's removal count from 68 to 91 but lowers Llama's from 90 to 87, and recovers calls poorly (event F1 .295 and .380 at IoU .5). Recognition stays well above the 20/120 reference in every arm (Table 11), so withholding boundaries changes which attack works rather than removing the leak. Because the representations differ, the gap from the call-window counts is not a pure causal effect of boundaries.

7.3. Planner shadows: consistent with the floor

The strongest tested call-window attack on shadow cover, a size/destination RF, scores 70/120 for Qwen and 64/120 for Llama; the validation-selected attacks score 54 and 64. On the realized test split, 21 of 120 shadow labels match the real one, so the idealized $k = 1$ floor of Section 5.3 is 70.5/120. The tested attacks are consistent with this floor: none is more accurate than an attacker who already knew both tasks would be. This does not show that the shadow reaches its privacy limit. The floor assumes that the real chain is exchangeable with the shadow, and real tickets are reused fixtures with their own cache histories, so a stronger role-identification attack could exceed it. Our chain-separation attack recovers the real chain exactly in only 29 and 32 fixtures, and among the 99 fixtures whose labels differ, where the floor is one half, aggregate counts bound the forest's real-label accuracy only to $[49, 70]/99$ and $[43, 64]/99$. An exact role test requires per-fixture predictions. Record-only attacks reach at most 73/120 for Qwen over 52 candidates, 2.5 fixtures above the floor; the selected attacks score 69 and 69. The prespecified equivalence family is degenerate for shadow because both selectors chose the same attack, and we do not count it.

What the shadow changes. The shadow arm differs from the controller arms in three ways at once. It adds more cover calls (8.14 and 10.46 tool calls per fixture for Qwen and Llama, against 5.6 and 6.8 for controller cover), it changes the composition of cover calls to those of a complete task, and it doubles model work (2.01 and 1.99 times the tokens). Its real-chain time rises from 2.91/3.33 s to 4.80/3.94 s, and call-window bytes rise 1.68/1.71 times. Our data compare the complete policy with its cost; they do not attribute the lower recognition to planner generation as such. Separating the factors needs a controller that replays recorded shadow chains with the same call count and composition but without a second planner, which we leave to future measurement (Section 8).

7.4. Goal completion misses state

In the live arms, all cover executes on an isolated clone. Goal and state counts stay within three fixtures of no cover for every arm (Table 9), although cover still changes planning: honest paths match no cover in only 115/120 (Qwen) and 103/120 (Llama) shadow fixtures. Llama's no-cover goal and state counts (82 and 70 of 120) are already imperfect.

TABLE 9. JOINT EVALUATION OF THE FIVE LIVE POLICIES; 120 TEST FIXTURES PER ROW, SINGLE HOST OVER LOOPBACK, SYNTHETIC TOOLS. SEL. S/A: VALIDATION-SELECTED SIZE-ONLY / ADAPTIVE ATTACK (PRIMARY). MAX: LARGEST TEST COUNT IN THE FROZEN SUITE (DESCRIPTIVE). GMM: LABEL-FREE REMOVAL. REF.: UNIFORM SIX-CLASS REFERENCE 20/120, OR FOR SHADOW THE REALIZED PAIR-AMBIGUITY FLOOR (TABLE 7). BYTES: WHOLE-CONNECTION / CALL-WINDOW FACTORS. TOKENS INCLUDE BOTH PLANNERS. GOAL AND STATE ARE SEPARATE COUNTS FROM THIS COHORT; THE STATE-CONTROL COHORT (SUPPLEMENT S1) HAS ITS OWN COUNTS.

Planner	Policy	Sel. S/A	Max	GMM	Ref.	Bytes×	Tokens×	Goal/State
Qwen3:8B	No cover	117/117	117	–	20	1.00/1.00	1.00	113/114
Qwen3:8B	Immediate	75/112	112	112	20	1.15/1.33	1.00	111/112
Qwen3:8B	History, sequential	76/93	99	53	20	1.15/1.32	1.00	112/113
Qwen3:8B	History, concurrent	66/99	99	38	20	1.15/1.33	1.00	111/112
Qwen3:8B	Planner shadow	54/54	70	34	70.5	1.31/1.68	2.01	112/113
Llama3.1:8B	No cover	110/110	112	–	20	1.00/1.00	1.00	82/70
Llama3.1:8B	Immediate	80/108	108	108	20	1.13/1.27	1.01	83/67
Llama3.1:8B	History, sequential	82/104	104	14	20	1.13/1.26	1.01	84/67
Llama3.1:8B	History, concurrent	68/106	106	66	20	1.13/1.27	1.02	82/67
Llama3.1:8B	Planner shadow	64/64	64	43	70.5	1.35/1.71	1.99	82/67

TABLE 10. ACHIEVED HISTORY-BASED COVER TIMING, ALL SPLITS (455 COVER CALLS PER ROW). MEANS IN SECONDS. AN ANCHOR GAP IS MEASURED FROM THE ACTUAL PRIOR COMPLETED ANCHOR CALL TO COVER RECORD START. SAMPLED DURATIONS ARE MINIMUM SERVER HOLDS, WHILE ACHIEVED DURATIONS INCLUDE RECORD-LEVEL TRANSPORT AND SCHEDULING.

Model	Arm	Drawn gap	Achieved anchor gap	Drawn hold	Achieved duration
Qwen	History seq.	0.5198	0.5660	0.0405	0.1004
Qwen	History concurrent	0.5198	0.5772	0.0405	0.1712
Llama	History seq.	0.5865	0.6203	0.0768	0.1097
Llama	History concurrent	0.5865	0.6204	0.0768	0.1234

TABLE 11. NO SUPPLIED CALL BOUNDARIES; WHOLE-EPISODE ASSOCIATION REMAINS SUPPLIED. B/R: VALIDATION-SELECTED IDLE-GRID BASELINE AND REMOVAL ATTACKS (PRIMARY PAIR: IMMEDIATE). MAX: LARGEST TEST COUNT OVER BOTH SUITES, INCLUDING THE EXPLORATORY PROTOCOL-AWARE EVENTIZER DESIGNED AFTER THE IDLE-GRID RESULTS WERE SEEN (DESCRIPTIVE). COUNTS OUT OF 120; UNIFORM REFERENCE 20.

	Qwen		Llama	
Policy	B/R	Max	B/R	Max
No cover	115/109	117	108/101	112
Immediate	80/69	91	90/86	97
History, sequential	73/74	75	88/56	93
History, concurrent	56/61	62	78/62	88
Planner shadow	69/65	73	69/56	69

The state control tests whether the evaluation would detect forbidden effects. Proxy, read and hidden-isolated cover keep Qwen state correctness at 115–116/120. Live writes reduce it to 7/120 while goal completion stays at 102/120: 95 of the 102 completed tasks leave collateral errors on other orders, shared quota or the ledger. Llama's counts are 3/120 intact and 73 of 76 completed tasks with collateral errors. Both integrity reductions survive Holm correction ($p = 3.7 \times 10^{-32}$ and 3.0×10^{-19}; Supplement S1). The harm direction is built into this stress arm; it shows what a goal-only evaluation omits, not how often production writes cause it.

7.5. Deciding with declared tolerances

Table 12 carries out step 4 of the protocol. The tolerances are illustrative: we set them for this synthetic scenario after the measurements, so the table demonstrates the procedure rather than validating any policy. Against the call-window observer every controller policy fails on inference, and the failures are robust: the lowest interval bound among them is .75. The shadow is the only policy not rejected, but its intervals, [.49, .67] for Qwen and [.44, .63] for Llama, straddle the 60% tolerance, so the protocol leaves it undecided; Llama's shadow also sits exactly at the state limit. The current sample is too small to decide. If the shadow's true accuracy equalled the 58.3% floor, a 95% interval would clear 60% only at roughly 3,400 test fixtures; with distinct label sampling, whose floor is 50%, an attack at that floor would already clear 60% at the current 120 fixtures.

The record-only column illustrates the protocol's same-suite requirement. Its suite lacks the context-removal and joint decoders that defeat history matching with call windows, so its weaker counts for Qwen's history arms do not show that withholding boundaries protects them; we mark those cells as not comparable. Where even the weaker suite exceeds the tolerance, as for all of Llama's controller arms, the failure stands. No policy exceeds the token, byte or latency tolerances. A latency tolerance of $1.5\times$ would fail Qwen's shadow ($1.65\times$), and a state tolerance of 2/120 would fail every Llama arm. Figure 3 places the measured policies against the derived floors.

TABLE 12. PROTOCOL STEP 4 ON THE LIVE POLICIES UNDER ILLUSTRATIVE TOLERANCES, SET BY US FOR THIS SYNTHETIC SCENARIO AFTER THE MEASUREMENTS: STRONGEST TESTED RECOGNITION $\leq 60\%$ (72/120); GOAL AND STATE EACH AT MOST 3/120 BELOW NO COVER; MODEL TOKENS $\leq 2.5\times$; CALL-WINDOW BYTES $\leq 2\times$; MEAN REAL-CHAIN LATENCY $\leq 2\times$. CW MAX CARRIES A 95% CLOPPER–PEARSON INTERVAL; IT IGNORES SELECTION OVER THE SUITE, SO IT IS TOO NARROW AND MAX MAY OVERSTATE A FRESHLY SELECTED ATTACK. INFERENCE PASSES ONLY IF THE INTERVAL LIES BELOW 60%, FAILS ONLY IF IT LIES ABOVE, AND IS OTHERWISE UNDECIDED. ΔG/ΔS ARE POINT DIFFERENCES FROM NO COVER. RO COUNTS COME FROM A SUITE WITHOUT CONTEXT REMOVAL OR JOINT DECODING, SO RO DECISIONS ON HISTORY ARMS ARE NOT COMPARABLE WITH CW DECISIONS.

Planner	Policy	CW Max [95% CI]	RO Max	ΔG/ΔS	Tok.×	Bytes×	Lat.×	Decision, CW observer	Decision, RO observer
Qwen3:8B	Immediate	112 [.87, .97]	91	−2/−2	1.00	1.33	1.23	Fail: inference	Fail: inference
Qwen3:8B	History, sequential	99 [.75, .89]	75	−1/−1	1.00	1.32	1.47	Fail: inference	Undecided; suite incomplete
Qwen3:8B	History, concurrent	99 [.75, .89]	62	−2/−2	1.00	1.33	1.14	Fail: inference	Undecided; suite incomplete
Qwen3:8B	Planner shadow	70 [.49, .67]	73	−1/−1	2.01	1.68	1.65	Undecided: inference	Undecided: inference
Llama3.1:8B	Immediate	108 [.83, .95]	97	+1/−3	1.01	1.27	0.88	Fail: inference	Fail: inference
Llama3.1:8B	History, sequential	104 [.79, .92]	93	+2/−3	1.01	1.26	1.15	Fail: inference	Fail: inference
Llama3.1:8B	History, concurrent	106 [.81, .94]	88	0/−3	1.02	1.27	0.84	Fail: inference	Fail: inference
Llama3.1:8B	Planner shadow	64 [.44, .63]	69	0/−3	1.99	1.71	1.18	Undecided: inference; state at limit	Undecided: inference

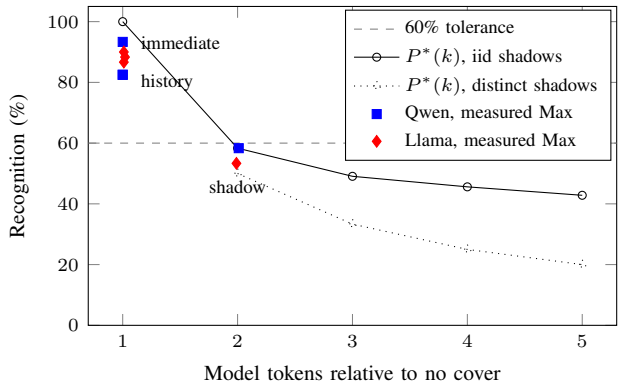


Figure 3. Inference versus model work against the call-window observer. Points are the strongest tested attacks for the four cover arms; lines are derived plausible-deniability floors for $k = 0$–4 shadows, not measurements. Only $k = 1$ with iid labels was measured. Controller cover costs no model work but leaves 83–93% recognition; the measured shadow sits on its floor.

8. Discussion

Design implications. Five lessons follow for executed cover. First, choose the dispatcher before the cover content: cover that does not pass through a model wait carries a timing signature that removal attacks exploit. Second, matching marginal gaps is insufficient; achieved timing and chronological context remain informative, so a schedule must be evaluated on the records an observer actually sees. Third, under exchangeability an idealized attacker on planner cover achieves $P^*(k)$, which falls slowly with more shadows and their linear token cost, whereas drawing shadow labels without replacement lowers it to $1/(k+1)$ at no extra cost; this is the cheapest improvement our analysis predicts and should be measured next. Fourth, state the service contract: the history arms rely on server-side holds and all arms on response padding, which a non-cooperating provider does not offer. Fifth, isolation is necessary but not sufficient: hidden isolated cover still changes planning paths, and only a state check against declared invariants detects collateral writes.

Threats to validity. The live study models a cooperating provider (Table 5); the non-cooperating case, with unpadded and unheld responses and cover against real state, is not measured. All runs are on one host over loopback, with synthetic tools, two fixed 8B models, and fixtures and phrasings reused across splits and studies. Loopback timing is cleaner than a wide-area path, so the timing attacks may be optimistic for a remote observer; network jitter and loss, cross-host deployment and unseen phrasings are untested. The recipient observer of Table 3, which also reads decoy arguments, is not measured, and the exact real-chain test for shadow requires per-fixture predictions not reported here. The shadow arm changes cover-call count, cover composition and model work together, so its lower recognition is not attributed to planner generation as such. Same-service calls serialize; unknown-task discovery and unrestricted multiplexing are not evaluated. The decision tolerances were set after the measurements, and the recognition intervals in Table 12 treat a suite maximum as a single attack; they ignore selection and are therefore too narrow. The record-only suite lacks the adaptive decoders used with call windows, so its counts are not comparable for history-matched cover. Persistent caches and phase order affect costs. The gap of Section 2.4 concerns four settings from two sources, one of them a preprint; twelve of the fifteen mapped studies are preprints.

Open measurements. The results call for six measurements: a controller that replays recorded shadow chains with matched call count and composition, to separate planner generation from cover volume; a shadow sweep over k and over iid versus distinct label sampling, with the exact real-chain test; context-removal and joint decoders on the record-only representation; the recipient view, using the per-service request journals; jitter, loss and cross-host deployments with unseen phrasings, including a non-cooperating provider; and equal-budget comparisons of controller and

planner cover under production state contracts with legitimate concurrent updates. Supplement S8 specifies further open questions on feedback attribution and task discovery.

9. Related Work

Agent side channels. Section 2 and Appendix A cover attacks on Agent traffic. Closest to our setting, WRA evaluates research decoys and destination visibility against a transit observer [2], and the communication-graph study evaluates metadata transformations and live OHTTP [3]; neither reports matched task utility for executed cover. Serving-side work shows that token streams, cache sharing and token counts leak through model interfaces [20], [12], [11], [13], [21], [14].

Decoy-query obfuscation. Private web search hides a user's queries among machine-generated decoys. TrackMeNot issues periodic decoy queries [63], and GooPIR submits each query together with $k-1$ decoy keywords to obtain k-fold deniability [67]. Peddinti and Saxena showed that classifiers separate TrackMeNot decoys from real queries [64], Balsa et al. systematized such systems and their failure to provide the deniability they claim [65], and Gervais et al. quantified the residual privacy of obfuscated search [68]. Our floor $P^*(k)$ is this deniability argument applied to complete Agent executions, and our dispatcher result parallels their lesson that decoys leak through the process that generates them. Two things are new in the Agent setting: decoys perform business work that must satisfy a state contract, and their generation is coupled to model inference, so a controller that bypasses inference creates a timing signature that query generators do not have.

Traffic-analysis defenses. WF defenses pad, pace and add dummy traffic, and their evaluation methodology, including adapted attacks and matched overheads, is well developed [15], [16], [32], [17], [18], [66], [69], [62]. Smart-home traffic shaping discards cover at a tunnel endpoint [34]. We carry their evaluation obligations over to cover that executes, and add business state and the dispatcher.

Agent security. Information-flow and privilege controls such as CaMeL, FIDES and Progent restrict what tools an Agent may call and what information flows between them [57], [58], [59]; CaMeL explicitly notes tool-call and timing side channels. SPILLage studies behavioral oversharing by browser Agents [27]. τ-bench and AgentDojo evaluate environment outcomes [60], [61]; our state endpoint follows their practice and binds it to traffic measurements. Surveys organize Agent threats by lifecycle and attack surface [53], [54], [55].

10. Ethics and Availability

Ethics. All experimental inputs are synthetic, and capture filters restrict collection to the experiment. The harmful live-write arm runs on isolated fixtures; no customers or production business states are affected. A hosted retest in Supplement S10 used account-holder authorization; its provider's research terms remain unverified, and it supports no claim in this paper. Traffic inference can enable profiling; our attacks target synthetic tickets only. AI tools assisted extraction, implementation and writing; every measurement is backed by saved observations and executable analysis. Account credentials, hosted-retest captures and transcripts, and third-party full texts are excluded from distribution.

Availability. The following are available to reviewers at [anonymized artifact link]: this paper and its supplement with LaTeX sources; the live-study collection and analysis code, workload and fixture generator, historical timing pools, frozen plans and their hashes; the record-only analysis code and frozen plans; aggregate results and test statistics for every reported table; the floor computation script; the forty-four-setting extraction and source locators; and the blind re-coding sheets, agreement script and adjudication record. These allow reviewers to inspect every implementation, recompute every reported test from the released aggregates and reproduce the floor values. Per-episode records, per-call timestamps and predictions, packet captures and TLS keys are retained locally and are not released; source hashes in the release manifests identify them. Consequently, refitting attacks or recomputing per-fixture quantities, such as the exact role test of Section 7.3, is not possible from the release.

11. Conclusion

Executed cover leaks through the process that dispatches it. Decoys inserted by a controller lack the model wait that precedes honest tool calls, and this difference is exploitable on real TLS records. Sampling historical gaps hides the shortcut but not the signal, and a shadow planner lowers recognition to a level consistent with its plausible-deniability reference at twice the model work, without yet isolating why. A goal check alone misses collateral state damage. Evaluating such cover requires binding inference, task completion, business state and cost to the same executions, and stating which service capabilities a policy assumes.

References

[1] Y. Zhang, X. Deng, Z. Gu, Y. Chen, K. Xu, Q. Li, and J. Wu, "Exposing LLM user privacy via traffic fingerprint analysis: A study of privacy risks in LLM agent interactions," *arXiv preprint arXiv:2510.07176*, 2025. [Online]. Available: https://arxiv.org/abs/2510.07176v1

[2] H. Jeong, M. Teymoorianfard, A. Kumar, A. Houmansadr, and E. Bagdasarian, "Network-level prompt and trait leakage in local research agents," in *35th USENIX Security Symposium*. USENIX Association, 2026, pp. 6485–6504. [Online]. Available: https://www.usenix.org/conference/usenixsecurity26/presentation/jeong

[3] B. Dangol, "From privacy to workflow integrity: Communication-graph metadata in autonomous agent interoperability," *arXiv preprint arXiv:2606.07150v3*, 2026. [Online]. Available: https://arxiv.org/abs/2606.07150v3

[4] S. Pouryousef, J. Lopez, S. R. Nowmi, M. M. Kamol, M. Hossain, M. Tran, and M. S. Rahman, "A large-scale benchmark and risk assessment of traffic analysis attacks on cloud LLM services," *arXiv preprint arXiv:2609.31877v2*, 2026. [Online]. Available: https://arxiv.org/abs/2609.31877v2

[5] C. Song, H. Wang, and R. Martin, "PersonaFingerprint: Measuring persona inference on modern websites with LLM-driven browsing," *arXiv preprint arXiv:2605.15962v1*, 2026. [Online]. Available: https://arxiv.org/abs/2605.15962v1

[6] D. Kang, H. Jeong, J. Sheffey, P. Datta, and A. Houmansadr, "Whose agent are you? multi-layer fingerprinting and attribution of autonomous web agents," *arXiv preprint arXiv:2606.20910v1*, 2026. [Online]. Available: https://arxiv.org/abs/2606.20910v1

[7] I. Fayolle, S. Bouhenniche, S. Pélissier, P. Laperdrix, C. Maurice, and W. Rudametkin, "On the internet, nobody knows you're an LLM bot: Unmasking web agents with multi-layer fingerprinting," *arXiv preprint arXiv:2606.30119v1*, 2026. [Online]. Available: https://arxiv.org/abs/2606.30119v1

[8] A. Shahriar, M. N. Rahman, S. Ahmed, F. Sadeque, and M. R. Parvez, "AgentTell: Behavioural side-channel leakage in browser-use agents," *arXiv preprint arXiv:2609.32915v1*, 2026. [Online]. Available: https://arxiv.org/abs/2609.32915v1

[9] F. Li, X. Gao, Z. Liu, T. Li, C. Fu, Z. Wang, and K. Xu, "ANT: A multi-granularity network traffic dataset and benchmark for agents behavior auditing," *arXiv preprint arXiv:2610.06514v1*, 2026. [Online]. Available: https://arxiv.org/abs/2610.06514v1

[10] D. Ayzenshteyn, R. Weiss, and Y. Mirsky, "Cloak, honey, trap: Proactive defenses against LLM agents," in *34th USENIX Security Symposium*. USENIX Association, 2025, pp. 8095–8114. [Online]. Available: https://www.usenix.org/conference/usenixsecurity25/presentation/ayzenshteyn

[11] L. Song, Z. Pang, W. Wang, Z. Wang, X. Wang, H. Chen, W. Song, Y. Jin, D. Meng, and R. Hou, "The early bird catches the leak: Unveiling timing side channels in LLM serving systems," *IEEE Transactions on Information Forensics and Security*, vol. 20, pp. 11 431–11 446, 2025, cited version: arXiv:2409.20002v5. [Online]. Available: https://ieeexplore.ieee.org/document/11206380/

[12] G. McDonald and J. B. Or, "Whisper leak: a side-channel attack on large language models," *arXiv preprint arXiv:2511.03675*, 2025, cited version: arXiv:2511.03675v1. [Online]. Available: https://arxiv.org/abs/2511.03675v1

[13] T. Zhang, G. Saileshwar, and D. Lie, "Time will tell: Timing side channels via output token count in large language models," *arXiv preprint arXiv:2412.15431*, 2024, cited version: arXiv:2412.15431v1. [Online]. Available: https://arxiv.org/abs/2412.15431v1

[14] G. Wu, Z. Li, Y. Zhang, Z. Zhang, J. Niu, Y. Wu, and Y. Zhang, "Cacheprune: Privacy-aware and fine-grained KV cache sharing for efficient LLM inference," *arXiv preprint arXiv:2605.23640v1*, 2026. [Online]. Available: https://arxiv.org/html/2605.23640v1

[15] M. Juarez, S. Afroz, G. Acar, C. Diaz, and R. Greenstadt, "A critical evaluation of website fingerprinting attacks," in *Proceedings of the ACM SIGSAC Conference on Computer and Communications Security*, 2014, pp. 263–274.

[16] N. Mathews, J. K. Holland, S. E. Oh, M. S. Rahman, N. Hopper, and M. Wright, "SoK: A critical evaluation of efficient website fingerprinting defenses," in *IEEE Symposium on Security and Privacy*, 2023.

[17] T. Wang and I. Goldberg, "Walkie-talkie: An efficient defense against passive website fingerprinting attacks," in *26th USENIX Security Symposium*, 2017. [Online]. Available: https://www.usenix.org/conference/usenixsecurity17/technical-sessions/presentation/wang-tao

[18] J. Gong and T. Wang, "Zero-delay lightweight defenses against website fingerprinting," in *29th USENIX Security Symposium*. USENIX Association, 2020, pp. 717–734. [Online]. Available: https://www.usenix.org/conference/usenixsecurity20/presentation/gong

[19] F. Tramèr, N. Carlini, W. Brendel, and A. Madry, "On adaptive attacks to adversarial example defenses," in *Advances in Neural Information Processing Systems*, vol. 33, 2020. [Online]. Available: https://arxiv.org/abs/2002.08347v2

[20] R. Weiss, D. Ayzenshteyn, G. Amit, and Y. Mirsky, "What was your prompt? a remote keylogging attack on ai assistants," in *33rd USENIX Security Symposium*, 2024. [Online]. Available: https://www.usenix.org/conference/usenixsecurity24/presentation/weiss

[21] N. Carlini and M. Nasr, "Remote timing attacks on efficient language model inference," *arXiv preprint arXiv:2410.17175v1*, 2024. [Online]. Available: https://arxiv.org/html/2410.17175v1

[22] D. Ahmed, A. Sabir, and A. Das, "Spying through your voice assistants: Realistic voice command fingerprinting," in *32nd USENIX Security Symposium*, 2023, pp. 2419–2436. [Online]. Available: https://www.usenix.org/conference/usenixsecurity23/presentation/ahmed-dilawer

[23] Model Context Protocol, "Transports," 2025, protocol version 2025-06-18. [Online]. Available: https://modelcontextprotocol.io/specification/2025-06-18/basic/transports

[24] M. Thomson and C. Benfield, "HTTP/2," RFC 9113, 2022. [Online]. Available: https://www.rfc-editor.org/rfc/rfc9113.html

[25] E. Rescorla, "The transport layer security (tls) protocol version 1.3," RFC 8446, 2018. [Online]. Available: https://www.rfc-editor.org/rfc/rfc8446.html

[26] W. Lugoloobi, S. Marro, J. Magomere, J. Wright, and C. Russell, "Known by their actions: Fingerprinting LLM browser agents via UI traces," *arXiv preprint arXiv:2605.14786v1*, 2026. [Online]. Available: https://arxiv.org/abs/2605.14786v1

[27] J. Roh, E. Bagdasarian, H. Haddadi, and A. S. Shamsabadi, "SPILLage: Agentic oversharing on the web," *arXiv preprint arXiv:2602.13516v1*, 2026. [Online]. Available: https://arxiv.org/abs/2602.13516v1

[28] S. Chen, R. Wang, X. Wang, and K. Zhang, "Side-channel leaks in web applications: A reality today, a challenge tomorrow," in *IEEE Symposium on Security and Privacy*, 2010. [Online]. Available: https://staff.ie.cuhk.edu.hk/~khzhang/my-papers/2010-oakland-side-channel.pdf

[29] P. Chapman and D. Evans, "Automated black-box detection of side-channel vulnerabilities in web applications," in *ACM Conference on Computer and Communications Security*, 2011. [Online]. Available: https://www.cs.virginia.edu/~evans/pubs/ccs2011/

[30] P. Reynolds, J. L. Wiener, J. C. Mogul, M. K. Aguilera, and A. Vahdat, "WAP5: Black-box performance debugging for wide-area systems," in *International World Wide Web Conference*, 2006. [Online]. Available: https://cseweb.ucsd.edu/~vahdat/papers/wap5-www06.pdf

[31] B. C. Tak, C. Tang, C. Zhang, S. Govindan, B. Urgaonkar, and R. N. Chang, "vPath: Precise discovery of request processing paths from black-box observations of thread and network activities," in *USENIX Annual Technical Conference*, 2009. [Online]. Available: https://www.usenix.org/conference/usenix-09/vpath-precise-discovery-request-processing-paths-black-box-observations-thread

[32] K. P. Dyer, S. E. Coull, T. Ristenpart, and T. Shrimpton, "Peek-a-boo, i still see you: Why efficient traffic analysis countermeasures fail," in *IEEE Symposium on Security and Privacy*, 2012. [Online]. Available: https://pages.cs.wisc.edu/~rist/papers/trafanal.pdf

[33] R. Trimananda, J. Varmarken, A. Markopoulou, and B. Demsky, "Packet-level signatures for smart home devices," in *Network and Distributed System Security Symposium*, 2020. [Online]. Available: https://www.ndss-symposium.org/ndss-paper/packet-level-signatures-for-smart-home-devices/

[34] N. Apthorpe, D. Y. Huang, D. Reisman, A. Narayanan, and N. Feamster, "Keeping the smart home private with smart(er) IoT traffic shaping," *Proceedings on Privacy Enhancing Technologies*, vol. 2019, no. 3, pp. 128–148, 2019. [Online]. Available: https://petsymposium.org/popets/2019/popets-2019-0040.pdf

[35] D. Duan, Q. Liu, Y. Liu, N. Zou, X. Hu, and D. Liu, "How much can tool calls reveal about you? privacy leakage from tool-call traces," *arXiv preprint arXiv:2610.41885v1*, 2026. [Online]. Available: https://arxiv.org/abs/2610.41885v1

[36] Y. Yao, Z. Wang, H. Cheng, Y. Cheng, H. Du, and X.-Y. Li, "IntentMiner: Intent inversion attack via tool call analysis in the model context protocol," *arXiv preprint arXiv:2512.14166v2*, 2026. [Online]. Available: https://arxiv.org/abs/2512.14166v2

[37] P. Sirinam, M. Imani, M. Juarez, and M. Wright, "Deep fingerprinting: Undermining website fingerprinting defenses with deep learning," in *Proceedings of the ACM SIGSAC Conference on Computer and Communications Security*, 2018. [Online]. Available: https://arxiv.org/abs/1801.02265v5

[38] S. Bhat, D. Lu, A. Kwon, and S. Devadas, "Var-CNN: A data-efficient website fingerprinting attack based on deep learning," *Proceedings on Privacy Enhancing Technologies*, vol. 2019, no. 4, pp. 292–310, 2019. [Online]. Available: https://petsymposium.org/popets/2019/popets-2019-0070.php

[39] M. Bishop, "HTTP/3," RFC 9114, 2022. [Online]. Available: https://www.rfc-editor.org/rfc/rfc9114.html

[40] J. Iyengar and M. Thomson, "QUIC: A UDP-based multiplexed and secure transport," RFC 9000, 2021. [Online]. Available: https://www.rfc-editor.org/rfc/rfc9000.html

[41] C. Song, D. D. M. Mekala, H. Wang, and R. P. Martin, "Redefining website fingerprinting attacks with multi-agent LLMs," *Proceedings on Privacy Enhancing Technologies*, vol. 2026, no. 3, pp. 688–702, 2026. [Online]. Available: https://www.petsymposium.org/popets/2026/popets-2026-0101.php

[42] S. Zavrak, "Content-aware attack detection in LLM agent tool-call traffic: An empirical study of features, architectures, and evaluation protocols," *arXiv preprint arXiv:2605.11053v3*, 2026. [Online]. Available: https://arxiv.org/abs/2605.11053v3

[43] Z. Guan, G. Xiong, G. Gou, Z. Li, M. Cui, and C. Liu, "BAPM: Block attention profiling model for multi-tab website fingerprinting attacks on Tor," in *Annual Computer Security Applications Conference*, 2021.

[44] X. Deng, Q. Yin, Z. Liu, X. Zhao, Q. Li, M. Xu, K. Xu, and J. Wu, "Robust multi-tab website fingerprinting attacks in the wild," in *IEEE Symposium on Security and Privacy*, 2023, pp. 1005–1022.

[45] T. Wang and I. Goldberg, "On realistically attacking Tor with website fingerprinting," *Proceedings on Privacy Enhancing Technologies*, vol. 2016, no. 4, pp. 21–36, 2016.

[46] J. Hayes and G. Danezis, "k-fingerprinting: A robust scalable website fingerprinting technique," in *25th USENIX Security Symposium*, 2016, pp. 1187–1203. [Online]. Available: https://www.usenix.org/conference/usenixsecurity16/technical-sessions/presentation/hayes

[47] G. Zhao, H. Wu, and X. Zhang, "AgentTrace: Execution-level fingerprinting of black-box LLM agents," 2026, version 1. [Online]. Available: https://arxiv.org/abs/2610.32092v1

[48] M. Ediga and S. Chattopadhyay, "Trace: Unmasking AI attack agents through terminal behavior fingerprinting," 2026, version 1. [Online]. Available: https://arxiv.org/abs/2605.01186v1

[49] O. Borysenko, "Developer experience with AI coding agents: HTTP behavioral signatures in documentation portals," 2026, extended version 3. [Online]. Available: https://arxiv.org/abs/2604.02544v3

[50] ——, "Documentation retrieval by AI coding agents: An empirical study of HTTP behavioral signatures," in *Proceedings of the 1st International Workshop on Agentic AI for Next-Generation Software Development*. ACM, 2026, pp. 168–169.

[51] Reworr and D. Volkov, "LLM agent honeypot: Monitoring AI hacking agents in the wild," 2025, version 2; first submitted 2024. [Online]. Available: https://arxiv.org/abs/2410.13919v2

[52] D. Arp, E. Quiring, F. Pendlebury, A. Warnecke, F. Pierazzi, C. Wressnegger, L. Cavallaro, and K. Rieck, "Dos and don'ts of machine learning in computer security," in *31st USENIX Security Symposium (USENIX Security 22)*. USENIX Association, 2022, pp. 3971–3988. [Online]. Available: https://www.usenix.org/conference/usenixsecurity22/presentation/arp

[53] Y. Yao, J. Duan, K. Xu, Y. Cai, Z. Sun, and Y. Zhang, "A survey on large language model (LLM) security and privacy: The good, the bad, and the ugly," *arXiv preprint arXiv:2312.02003v3*, 2024. [Online]. Available: https://arxiv.org/abs/2312.02003v3

[54] A. Shahriar, M. N. Rahman, S. Ahmed, F. Sadeque, and M. R. Parvez, "A survey on agentic security: Applications, threats and defenses," *arXiv preprint arXiv:2510.06445v1*, 2025. [Online]. Available: https://arxiv.org/abs/2510.06445v1

[55] M. A. Ferrag, N. Tihanyi, D. Hamouda, L. Maglaras, and M. Debbah, "From prompt injections to protocol exploits: Threats in LLM-powered AI agents workflows," *arXiv preprint arXiv:2506.23260v1*, 2025. [Online]. Available: https://arxiv.org/abs/2506.23260v1

[56] LangChain, "LangGraph, version 1.0.3," Software release, November 10, 2025. [Online]. Available: https://pypi.org/project/langgraph/1.0.3/

[57] E. Debenedetti, I. Shumailov, T. Fan, J. Hayes, N. Carlini, D. Fabian, C. Kern, C. Shi, A. Terzis, and F. Tramèr, "Defeating prompt injections by design," 2025, version 2. [Online]. Available: https://arxiv.org/abs/2503.18813v2

[58] M. Costa, B. Köpf, A. Kolluri, A. Paverd, M. Russinovich, A. Salem, S. Tople, L. Wutschitz, and S. Zanella-Béguelin, "Securing AI agents with information-flow control," 2025, version 2. [Online]. Available: https://arxiv.org/abs/2505.23643v2

[59] T. Shi, J. He, Z. Wang, H. Li, L. Wu, W. Guo, and D. Song, "Progent: Programmable privilege control for LLM agents," 2025, version 2. [Online]. Available: https://arxiv.org/abs/2504.11703v2

[60] S. Yao, N. Shinn, P. Razavi, and K. Narasimhan, "τ-bench: A benchmark for tool-agent-user interaction in real-world domains," 2024, version 1. [Online]. Available: https://arxiv.org/abs/2406.12045v1

[61] E. Debenedetti, J. Zhang, M. Balunović, L. Beurer-Kellner, M. Fischer, and F. Tramèr, "AgentDojo: A dynamic environment to evaluate prompt injection attacks and defenses for LLM agents," in *Advances in Neural Information Processing Systems*, vol. 37, 2024. [Online]. Available: https://proceedings.neurips.cc/paper_files/paper/2024/hash/97091a5177d8dc64b1da8bf3e1f6fb54-Abstract-Datasets_and_Benchmarks_Track.html

[62] X. Cai, R. Nithyanand, T. Wang, R. Johnson, and I. Goldberg, "A systematic approach to developing and evaluating website fingerprinting defenses," in *Proceedings of the ACM SIGSAC Conference on Computer and Communications Security*, 2014, pp. 227–238.

[63] D. C. Howe and H. Nissenbaum, "TrackMeNot: Resisting surveillance in web search," in *Lessons from the Identity Trail: Anonymity, Privacy, and Identity in a Networked Society*. Oxford University Press, 2008, pp. 417–436.

[64] S. T. Peddinti and N. Saxena, "On the privacy of web search based on query obfuscation: A case study of TrackMeNot," in *Privacy Enhancing Technologies (PETS)*, ser. LNCS, vol. 6205. Springer, 2010, pp. 19–37.

[65] E. Balsa, C. Troncoso, and C. Diaz, "OB-PWS: Obfuscation-based private web search," in *IEEE Symposium on Security and Privacy*, 2012, pp. 491–505.

[66] M. Juarez, M. Imani, M. Perry, C. Diaz, and M. Wright, "Toward an efficient website fingerprinting defense," in *European Symposium on Research in Computer Security (ESORICS)*, ser. LNCS, vol. 9878. Springer, 2016, pp. 27–46.

[67] J. Domingo-Ferrer, A. Solanas, and J. Castellà-Roca, "h(k)-private information retrieval from privacy-uncooperative queryable databases," *Online Information Review*, vol. 33, no. 4, pp. 720–744, 2009.

[68] A. Gervais, R. Shokri, A. Singla, S. Capkun, and V. Lenders, "Quantifying web-search privacy," in *ACM SIGSAC Conference on Computer and Communications Security (CCS)*, 2014, pp. 966–977.

[69] V. Shmatikov and M.-H. Wang, "Timing analysis in low-latency mix networks: Attacks and defenses," in *European Symposium on Research in Computer Security (ESORICS)*, ser. LNCS, vol. 4189. Springer, 2006, pp. 18–33.

Appendix A.
Map of Agent Communication Side-Channel Studies

Inclusion and retrieval. A study is included if an LLM selects actions with external feedback able to affect later actions, and the study links that execution to a communication-side inference or an evaluated protection. Client, tool, model and inter-agent interfaces are eligible; transit, recipient and active observers remain separate. Whole semantic trajectories or browser instrumentation alone, deliberate covert messages and prompt-injection defenses without a communication-side endpoint are excluded [36], [26]. The window covers first public submissions from 1 January 2024 through 9 November 2026. Twelve topic queries, a native arXiv query (180 records) and public-web queries (33 scholarly candidates) gave 209 records after deduplication; two citation leads brought screening to 211, of which fourteen were direct, forty adjacent, 155 outside scope and two unresolved. A retained published seed (CHeaT) gives fifteen direct sources with forty-four settings. Twelve of the fifteen lack a verified formal publication at the cutoff. Exact queries and dispositions are in Supplement S4.

Coding and reliability. Each setting record binds source version and locator to access, features, learner, target, population, split, intervention, result and cost. We assigned and decided every code against the cited passage; AI tools located candidate passages and proposed drafts, which we accepted, changed or rejected. An independent security PhD researcher, blind to the manuscript and our codes, re-coded the decision-critical subset from the sources without AI tools. Access class (44/44), intervention purpose (9/9), and adaptation and utility in the four privacy settings (4/4 each) agree fully ($\kappa = 1$); the protection flag agrees on 38/44 ($\kappa = .61$) and transfer relations on 5/6 ($\kappa = .77$). After adjudication of the seven disagreements against the sources, no privacy setting has both an adapted attacker and matched utility (Supplement S6.1).

The map. Supplement S11 lists all forty-four settings with their interface, access, target, signal, unit, reported result and intervention evidence. Twenty-two use transit observations, ten recipient access, eleven active access and one inferred preferences. Three settings compare identity obfuscation and two adversarial-Agent blocking; thirty-four contain no protection comparison. Scores keep their source-defined metric and target and are not cross-paper rankings. Table 13 relates the Agent evidence to established traffic-analysis conditions: a condition is *retained* when it applies directly, *extended* when an execution condition must be added, *untested* when the Agent evidence does not evaluate it, and an *access mismatch* when host instrumentation is mistaken for passive observation.

Appendix B.
Defense Design Space

Table 14 lists defense layers, what each requires the defender to control, the costs to measure together and the exposure that remains.

TABLE 13. TRANSFER OF ESTABLISHED CONDITIONS TO AGENT EXECUTION. RELATIONSHIP LABELS CONCERN THE STATED MAPPING; EVIDENCE STRENGTH AND MISSING TESTS APPEAR SEPARATELY. A MISSING AGENT TEST DOES NOT REFUTE THE CLASSICAL RESULT.

Established relation and condition	Agent mapping / consequence	Relation	Agent evidence and remaining test
Unknown boundaries and background affect event recognition [45], [43], [44], [33].	Tool bursts, model waits and task intervals are different units. Evaluate proposed events with false alarms as well as classification.	Extended	ANT supplies stage boundaries; the concurrent feedback study (Supplement S10) tests particular detectors. Greater Agent localization difficulty is untested.
Defense evaluation must allow adaptation to the protected distribution [32], [16], [19].	Record protected training access and still-visible links; an unchanged learner family can refit.	Retained	Transformed and live OHTTP settings report adaptation in one preprint [3]; WRA's matched protected examples remain unresolved.
Coarse timing/volume can remain useful under specified padding [32]; serving streams expose length/gap signals [20], [12].	Name the shaping policy, receiving hop and Agent workload before carrying over a measured effect.	Untested generalization	Cloud and ANT measure model-view recognition [4], [9]. A general protected-Agent SSE claim is not tested by these observations.
An observation result is tied to the visible paths and representations [28], [34].	Feedback forwards tool information into later model requests across separately controlled services; evaluate every permitted residual view.	Extended	Published WRA tool-side evidence and preprint model/recipient results establish different exposed interfaces. Causal feedback attribution needs its own test.
STP tunnel endpoints remove cover before it reaches devices or servers [34, Sec. 7.1].	When cover executes as research, add original-task resource, state and feedback constraints to the traffic budget.	Extended	Published WRA reports executed research decoys and utility under a different intervention [2]. Our controlled LLM case separates task completion from state integrity (Section 7.4); it does not evaluate WRA writes.
Host traces can reconstruct distributed request paths [30], [31].	Instrumentation can provide ground truth; a transit attacker must infer from its own permitted observations.	Access mismatch	ANT stage labels and local task intervals are evaluator-provided. Host tracing alone cannot establish passive encrypted-stream localization.

TABLE 14. DEFENSE DESIGN SPACE: ENFORCEMENT AUTHORITY, COSTS TO MEASURE AND REMAINING EXPOSURE. SOURCE-SPECIFIC MEASUREMENTS APPEAR IN THE TEXT AND SUPPLEMENT S7.

Layer / mechanism	What must be trusted or controlled	Costs to measure together	Residual exposure and practical decision
Naming / association: encrypted name resolution, relays, OHTTP	Name resolver or relay placement; the parties able to link the user to the destination	Added hops, bytes, latency and availability	Shape can survive name hiding. Choose this layer when destination association is the private target; then test remaining observers.
Representation: message, TLS-record or packet padding	Component that controls the final visible size; padding survives later encoding	Added bytes, record/packet count, fragmentation	Timing, count and other links remain. State the padding layer and test downstream captures.
Scheduling: batching, pacing, fixed-rate cover	Sender/relay controls output schedule and cover lifetime	First-response delay, completion delay, bandwidth during idle periods	Admission, termination and overflow can reveal activity. Budget these events as part of the visible trace.
Transport cover: dummy exchanges discarded at a tunnel endpoint	Trusted cover terminator; no dispatch into business tools	Cover bandwidth, queueing, resource use	Other endpoints still see their inputs. Prefer this boundary when the aim is traffic shaping without extra business work.
Execution cover: extra research or tool calls	Tool privileges, isolated state and whether decoy feedback reaches the planner	Model tokens, tool quota, task success, state changes and latency	A decoy can affect the real execution. Measure privacy and original-task utility on the same protected runs.
Shared-state isolation / minimization	Tenant separation, cache keys and information passed between services	Lost cache reuse, compute, latency, task accuracy	Isolation removes a sharing channel, not every traffic signal. Minimize tool arguments and feedback according to each recipient's role.